DATABASE MANAGEMENT SYSTEM

Advanced SQL: Joins & Subqueries

Multi-table reports for the online_shop database

Submitted By	
Name	Palle Sai Rithesh
Roll Number	2511021240013
Course	BCA Game Development
Subject	Database Management System
Assignment	Assignment 9 — Advanced SQL: Joins & Subqueries
Project	Online Shopping System

Assignment 9 — requirements

- Develop advanced SQL queries using appropriate joins and subqueries.
- Generate meaningful reports relevant to your proposed system.
- Include reports involving multiple tables, summary statistics, and related records based on your project's requirements.
- Provide the SQL queries, corresponding outputs, and brief explanations for each report.

01 OVERVIEW

Reports, Joins and Subqueries

Nineteen reports built on the ten-table online_shop database and the data from Assignment 7.

A relational design spreads facts over many tables, so useful reports need joins and subqueries to put them back together. The reports below are grouped by technique, and each one answers a question a shop owner would really ask.

Technique	When it is the right tool	Reports
INNER JOIN	Combine rows that have a match on both sides	1.1, 1.2, 1.3 and all of section 2
LEFT JOIN	Keep every row of the left table, even without a match; find “missing” rows with IS NULL	1.4, 1.5, 1.6, 2.4, 4.2
Scalar subquery	The inner query returns one value to compare with or display	3.1, 3.6, 4.3
IN subquery	Match against a list produced by another query	3.2
Correlated subquery	The inner query depends on the current outer row	3.3, 4.3
EXISTS / NOT EXISTS	Test whether related rows are present or absent	3.4, 3.5
Derived table	A grouped query used like a table in FROM	2.4, 3.6

Note: All statements are standard SQL and run unchanged on MySQL 8. Table aliases (o, c, p and so on) keep long joins readable. Outputs come from executing each statement on the Assignment 7 data.

02 TASK 1

Reports Using Joins

Related records from several tables brought together.

Report 1.1 Order summary with customer names

INNER JOIN: only orders that have a matching customer are returned.

```
SELECT o.order_id, c.name AS customer, o.order_date, o.status, o.total_amount
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
ORDER BY o.order_id;
```

OUTPUT

order_id	customer	order_date	status	total_amount
1	Aarav Sharma	2026-08-03 10:15:00	Completed	2098.00
2	Priya Nair	2026-08-07 14:40:00	Completed	17098.00
3	Rohan Mehta	2026-08-12 09:05:00	Completed	2796.00
4	Ananya Iyer	2026-08-19 18:22:00	Completed	3198.00
5	Karthik Reddy	2026-08-25 21:10:00	Completed	7298.00
6	Aarav Sharma	2026-09-02 11:30:00	Completed	2598.00
7	Sneha Patil	2026-09-06 16:45:00	Completed	2448.00
8	Vikram Singh	2026-09-10 12:00:00	Cancelled	3299.00
9	Divya Menon	2026-09-14 19:35:00	Completed	3797.00
10	Priya Nair	2026-09-20 13:15:00	Completed	5497.00
11	Arjun Das	2026-09-27 10:50:00	Confirmed	4398.00
12	Meera Kulkarni	2026-10-03 15:25:00	Confirmed	4149.00
13	Rohan Mehta	2026-10-09 20:05:00	Placed	3197.00
14	Karthik Reddy	2026-10-15 08:40:00	Placed	18998.00
15	Imran Khan	2026-10-22 17:55:00	Placed	4997.00

(15 rows)

Explanation: Each of the 15 orders is matched to its customer through customer_id. Four customers (Aarav, Priya, Rohan and Karthik) appear twice because they placed two orders each.

Report 1.2 Order line report for the first five orders

Five tables joined: order lines, orders, customers, products and categories.

```
SELECT o.order_id, c.name AS customer, p.name AS product, cat.category_name AS category,
       i.quantity, i.quantity * i.unit_price AS line_amount
FROM order_items i
JOIN orders     o   ON o.order_id     = i.order_id
JOIN customers  c   ON c.customer_id  = o.customer_id
JOIN products   p   ON p.product_id   = i.product_id
JOIN categories cat ON cat.category_id = p.category_id
WHERE o.order_id <= 5
ORDER BY o.order_id, i.order_item_id;
```

OUTPUT

order_id	customer	product	category	quantity	line_amount
1	Aarav Sharma	Wireless Bluetooth Earbuds	Electronics	1	1499.00
1	Aarav Sharma	SQL Fundamentals for Beginners	Books	1	599.00
2	Priya Nair	27-inch IPS Monitor	Electronics	1	15499.00
2	Priya Nair	Gaming Mouse	Gaming	1	1599.00
3	Rohan Mehta	Cotton Round-Neck T-Shirt	Clothing	3	1497.00
3	Rohan Mehta	Slim Fit Denim Jeans	Clothing	1	1299.00
4	Ananya Iyer	Non-Stick Cookware Set	Home & Kitchen	1	2299.00
4	Ananya Iyer	Electric Kettle 1.5 L	Home & Kitchen	1	899.00
5	Karthik Reddy	Wireless Game Controller	Gaming	1	3299.00
5	Karthik Reddy	Mechanical Gaming Keyboard	Gaming	1	3999.00

(10 rows)

Explanation: Joining five tables turns foreign-key numbers into readable names. The line amount uses the unit_price stored in the order line, so order 2 shows ₹15,499.00 for the monitor even though today's price is ₹14,999.00.

Report 1.3 Delivery addresses of orders still being processed

Orders joined to customers, addresses and pincodes to print a full delivery location.

```
SELECT o.order_id, c.name AS customer, a.house_street, pc.city, pc.state, pc.pincode
FROM orders o
JOIN customers c  ON c.customer_id = o.customer_id
JOIN addresses a  ON a.address_id  = o.address_id
JOIN pincodes   pc ON pc.pincode    = a.pincode
WHERE o.status IN ('Placed', 'Confirmed')
ORDER BY o.order_id;
```

OUTPUT

order_id	customer	house_street	city	state	pincode
11	Arjun Das	22, Navrangpura Society	Ahmedabad	Gujarat	380001
12	Meera Kulkarni	5, Koregaon Park Lane	Pune	Maharashtra	411001
13	Rohan Mehta	203, Marine Lines Apartments	Mumbai	Maharashtra	400001
14	Karthik Reddy	14, Jubilee Hills Road No. 5	Hyderabad	Telangana	500001
15	Imran Khan	101, Park Street	Kolkata	West Bengal	700001

(5 rows)

Explanation: Following orders → addresses → pincodes gives a complete delivery location. City and state come from the pincodes table, where they are stored only once.

Report 1.4 Customers who have never placed an order

LEFT JOIN keeps every customer; rows with no matching order show NULL.

```
SELECT c.customer_id, c.name, c.email
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

OUTPUT

customer_id	name	email
12	Lakshmi Venkatesh	lakshmi.venkatesh@example.com

(1 row)

Explanation: A LEFT JOIN keeps every customer; where no order matches, the order columns are NULL. Filtering on *o.order_id IS NULL* leaves only customers without orders, here Lakshmi Venkatesh. An INNER JOIN would have dropped her.

Report 1.5 Products that have never been ordered

The same LEFT JOIN pattern applied to the catalogue.

```
SELECT p.product_id, p.name, p.price, p.stock_qty
FROM products p
LEFT JOIN order_items i ON i.product_id = p.product_id
WHERE i.order_item_id IS NULL;
```

OUTPUT

product_id	name	price	stock_qty
20	Large Mouse Pad	399.00	60

(1 row)

Explanation: The same pattern finds products that no order has ever contained. Large Mouse Pad (60 units in stock) is a candidate for a promotion.

Report 1.6 Payment and shipment status of selected orders

Two LEFT JOINs show orders even when a payment or shipment is missing.

```
SELECT o.order_id, o.status AS order_status, y.payment_method, y.payment_status,
       s.carrier, s.delivery_status
FROM orders o
LEFT JOIN payments  y ON y.order_id = o.order_id
LEFT JOIN shipments s ON s.order_id = o.order_id
WHERE o.order_id IN (6, 8, 13, 14)
ORDER BY o.order_id, y.payment_id, s.shipment_id;
```

OUTPUT

order_id	order_status	payment_method	payment_status	carrier	delivery_status
6	Completed	UPI	Paid	Delhivery	Delivered
6	Completed	UPI	Paid	India Post	Delivered
8	Cancelled	UPI	Refunded	NULL	NULL
13	Placed	COD	Pending	NULL	NULL
14	Placed	Card	Failed	NULL	NULL
14	Placed	UPI	Paid	NULL	NULL

(6 rows)

Explanation: Order 6 appears twice because it was split across two shipments, and order 14 appears twice because it has two payment attempts (Card failed, UPI paid). Orders 8, 13 and 14 have no shipment, so carrier and delivery status are NULL. An INNER JOIN would have hidden those orders.

03 TASK 2

Summary Statistics Across Tables

Joins combined with GROUP BY and aggregate functions.

Report 2.1 Sales by product category

Summary statistics over four joined tables; cancelled orders are excluded.

```
SELECT cat.category_name, SUM(i.quantity) AS units_sold,
       SUM(i.quantity * i.unit_price) AS revenue
FROM order_items i
JOIN orders     o   ON o.order_id      = i.order_id
JOIN products   p   ON p.product_id    = i.product_id
JOIN categories cat ON cat.category_id = p.category_id
WHERE o.status <> 'Cancelled'
GROUP BY cat.category_name
ORDER BY revenue DESC;
```

OUTPUT

category_name	units_sold	revenue
Electronics	9	42791.00
Gaming	6	17794.00
Home & Kitchen	4	7596.00
Clothing	7	5593.00
Sports & Fitness	4	4896.00
Books	5	3897.00

(6 rows)

Explanation: Electronics earns the most (₹42,791.00 from nine units), more than twice Gaming's ₹17,794.00. Cancelled order 8 is excluded, so Sports & Fitness counts four units. Books sells five units but earns the least.

Report 2.2 Top five customers by spend

Join, group, aggregate and limit to build a leaderboard.

```
SELECT c.customer_id, c.name, COUNT(*) AS order_count, SUM(o.total_amount) AS total_spend
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status <> 'Cancelled'
GROUP BY c.customer_id, c.name
ORDER BY total_spend DESC
LIMIT 5;
```

OUTPUT

customer_id	name	order_count	total_spend
5	Karthik Reddy	2	26296.00
2	Priya Nair	2	22595.00
3	Rohan Mehta	2	5993.00
11	Imran Khan	1	4997.00
1	Aarav Sharma	2	4696.00

(5 rows)

Explanation: Karthik Reddy leads with ₹26,296.00 over two orders, followed by Priya Nair with ₹22,595.00; both orders include a monitor. DESC puts the biggest spender first and LIMIT 5 keeps the top five.

Report 2.3 Sales by city

Orders are traced through addresses to pincodes to reach the city.

```
SELECT pc.city, COUNT(*) AS order_count, SUM(o.total_amount) AS sales_amount
FROM orders o
JOIN addresses a  ON a.address_id = o.address_id
JOIN pincodes  pc ON pc.pincode   = a.pincode
WHERE o.status <> 'Cancelled'
GROUP BY pc.city
ORDER BY sales_amount DESC, pc.city;
```

OUTPUT

city	order_count	sales_amount
Bengaluru	4	27291.00
Hyderabad	2	26296.00
Kolkata	2	8794.00
Pune	2	6597.00
Mumbai	2	5993.00
Ahmedabad	1	4398.00
Chennai	1	3198.00

(7 rows)

Explanation: Bengaluru is the strongest city (four orders, ₹27,291.00). Hyderabad is second with only two orders because one of them, the monitor with a keyboard, is large. The city name is reached by following order → address → pincode.

Report 2.4 Product popularity and rating report

Two grouped subqueries in the FROM clause are LEFT JOINed to the catalogue.

```
SELECT p.product_id, p.name,
       COALESCE(s.units_sold, 0)   AS units_sold,
       COALESCE(r.review_count, 0) AS review_count,
       r.avg_rating
FROM products p
LEFT JOIN (SELECT i.product_id, SUM(i.quantity) AS units_sold
           FROM order_items i JOIN orders o ON o.order_id = i.order_id
           WHERE o.status <> 'Cancelled'
           GROUP BY i.product_id) s ON s.product_id = p.product_id
LEFT JOIN (SELECT product_id, COUNT(*) AS review_count, ROUND(AVG(rating), 2) AS avg_rating
           FROM reviews GROUP BY product_id) r ON r.product_id = p.product_id
ORDER BY units_sold DESC, p.product_id;
```

OUTPUT

product_id	name	units_sold	review_count	avg_rating
5	Cotton Round-Neck T-Shirt	5	2	3.50
1	Wireless Bluetooth Earbuds	4	2	3.50
8	SQL Fundamentals for Beginners	3	2	4.50
14	Yoga Mat 6 mm	3	0	NULL
2	20000 mAh Power Bank	2	1	4.00
4	27-inch IPS Monitor	2	1	5.00
12	Electric Kettle 1.5 L	2	0	NULL
17	Wireless Game Controller	2	1	4.00
18	Mechanical Gaming Keyboard	2	1	5.00
19	Gaming Mouse	2	1	4.00
3	Smart Fitness Band	1	1	3.00
6	Slim Fit Denim Jeans	1	0	NULL
7	Hooded Sweatshirt	1	0	NULL
9	Database System Concepts	1	1	5.00
10	Introduction to Game Design	1	0	NULL
11	Non-Stick Cookware Set	1	1	5.00
13	Mixer Grinder 750 W	1	0	NULL
16	Running Shoes	1	1	4.00
15	Adjustable Dumbbell Pair	0	0	NULL
20	Large Mouse Pad	0	0	NULL

(20 rows)

Explanation: The two subqueries are summed separately and then joined to the catalogue. Joining order lines and reviews directly would multiply rows and inflate the counts. LEFT JOIN and COALESCE keep products that have no sales or reviews, and avg_rating stays NULL for products nobody has reviewed.

04 TASK 3

Reports Using Subqueries

A query nested inside another query.

Report 3.1 Products priced above the catalogue average

Scalar subquery: the inner query returns one value that the outer query compares against.

```
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

OUTPUT

name	price
27-inch IPS Monitor	14999.00
Mechanical Gaming Keyboard	3999.00
Mixer Grinder 750 W	3499.00
Adjustable Dumbbell Pair	3299.00
Wireless Game Controller	3299.00
Running Shoes	2799.00

(6 rows)

Explanation: The inner query returns one number, the average price ₹2,499.10, and the outer query keeps products above it. Smart Fitness Band at ₹2,499.00 misses by ten paise.

Report 3.2 Customers who bought a Gaming product

IN subquery: the inner query returns a list of customer ids.

```
SELECT customer_id, name
FROM customers
WHERE customer_id IN (SELECT o.customer_id
                      FROM orders o
                      JOIN order_items i ON i.order_id   = o.order_id
                      JOIN products    p ON p.product_id = i.product_id
                      WHERE p.category_id = 6)
ORDER BY customer_id;
```

OUTPUT

customer_id	name
2	Priya Nair
5	Karthik Reddy
10	Meera Kulkarni
11	Imran Khan

(4 rows)

Explanation: The inner query builds a list of customer ids who bought a Gaming product (customer 5 bought several but is listed once by IN). The outer query turns the ids into names.

Report 3.3 Products priced above their own category's average

Correlated subquery: the inner query is re-evaluated for each outer row.

```
SELECT p.product_id, p.category_id, p.name, p.price
FROM products p
WHERE p.price > (SELECT AVG(p2.price)
                 FROM products p2
                 WHERE p2.category_id = p.category_id)
ORDER BY p.category_id, p.product_id;
```

OUTPUT

product_id	category_id	name	price
4	1	27-inch IPS Monitor	14999.00
6	2	Slim Fit Denim Jeans	1299.00
7	2	Hooded Sweatshirt	1799.00
9	3	Database System Concepts	1250.00
11	4	Non-Stick Cookware Set	2299.00
13	4	Mixer Grinder 750 W	3499.00
15	5	Adjustable Dumbbell Pair	3299.00
16	5	Running Shoes	2799.00
17	6	Wireless Game Controller	3299.00
18	6	Mechanical Gaming Keyboard	3999.00

(10 rows)

Explanation: The inner query is re-run for every product and averages only that product's own category. Books average ₹899.67, so only Database System Concepts (₹1,250.00) qualifies from that category.

Report 3.4 Orders that had a failed payment attempt

EXISTS is true when the inner query finds at least one row.

```
SELECT o.order_id, o.customer_id, o.total_amount
FROM orders o
WHERE EXISTS (SELECT 1 FROM payments y
              WHERE y.order_id = o.order_id AND y.payment_status = 'Failed');
```

OUTPUT

order_id	customer_id	total_amount
14	5	18998.00

(1 row)

Explanation: EXISTS only asks whether at least one matching row is present. Order 14 is the only order that had a failed payment attempt.

Report 3.5 Paid orders that have not been shipped yet

EXISTS combined with NOT EXISTS gives the warehouse its to-do list.

```
SELECT o.order_id, o.status, o.total_amount
FROM orders o
WHERE EXISTS (SELECT 1 FROM payments y
              WHERE y.order_id = o.order_id AND y.payment_status = 'Paid')
  AND NOT EXISTS (SELECT 1 FROM shipments s WHERE s.order_id = o.order_id);
```

OUTPUT

order_id	status	total_amount
14	Placed	18998.00

(1 row)

Explanation: Order 14 is paid (second attempt, UPI) but has no shipment row yet, so it is the next order to pack. Orders 13 and 15 are not listed because their payments are still pending.

Report 3.6 Customers whose average order beats the overall average

A derived table in FROM plus a scalar subquery in WHERE.

```
SELECT c.name, t.order_count, t.avg_order_value
FROM (SELECT customer_id, COUNT(*) AS order_count,
             ROUND(AVG(total_amount), 2) AS avg_order_value
      FROM orders
      WHERE status <> 'Cancelled'
      GROUP BY customer_id) t
JOIN customers c ON c.customer_id = t.customer_id
WHERE t.avg_order_value > (SELECT AVG(total_amount) FROM orders WHERE status <> 'Cancelled')
ORDER BY t.avg_order_value DESC;
```

OUTPUT

name	order_count	avg_order_value
Karthik Reddy	2	13148.00
Priya Nair	2	11297.50

(2 rows)

Explanation: The derived table calculates each customer's average order and the scalar subquery supplies the overall average (₹5,897.64 over 14 non-cancelled orders). Only Karthik Reddy and Priya Nair are above it.

05 TASK 4

Management Reports

Combined techniques for day-to-day decisions.

Report 4.1 Carrier delivery performance

Conditional counting with CASE inside an aggregate.

```
SELECT carrier, COUNT(*) AS shipment_count,
       SUM(CASE WHEN delivery_status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_count
FROM shipments
GROUP BY carrier
ORDER BY shipment_count DESC, carrier;
```

OUTPUT

carrier	shipment_count	delivered_count
Delhivery	5	4
Blue Dart	3	3
Ecom Express	3	2
India Post	1	1

(4 rows)

Explanation: Delhivery carried the most shipments (five); four are delivered and order 12 is still packed. Blue Dart delivered all three of its shipments. CASE inside SUM counts only the rows that meet a condition.

Report 4.2 Low-stock alert with demand

Products with fewer than 50 units left, together with the units customers have ordered.

```
SELECT p.product_id, p.name, p.stock_qty, COALESCE(SUM(i.quantity), 0) AS units_ordered
FROM products p
LEFT JOIN order_items i ON i.product_id = p.product_id
WHERE p.stock_qty < 50
GROUP BY p.product_id, p.name, p.stock_qty
ORDER BY p.stock_qty, p.product_id;
```

OUTPUT

product_id	name	stock_qty	units_ordered
10	Introduction to Game Design	0	1
4	27-inch IPS Monitor	25	2
15	Adjustable Dumbbell Pair	30	1
13	Mixer Grinder 750 W	35	1
18	Mechanical Gaming Keyboard	38	2
17	Wireless Game Controller	45	2

(6 rows)

Explanation: Six products have fewer than 50 units left. Introduction to Game Design is out of stock yet already ordered, so it should be restocked first. The units_ordered column includes the cancelled dumbbell order.

Report 4.3 Customer overview report

Scalar subqueries in the SELECT list collect facts from three tables for every customer.

```
SELECT c.customer_id, c.name,
  (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) AS order_count,
  (SELECT COALESCE(SUM(o.total_amount), 0) FROM orders o
   WHERE o.customer_id = c.customer_id AND o.status <> 'Cancelled') AS total_spend,
  (SELECT COUNT(*) FROM reviews r WHERE r.customer_id = c.customer_id) AS review_count,
  (SELECT MAX(o.order_date) FROM orders o WHERE o.customer_id = c.customer_id) AS last_order
FROM customers c
ORDER BY total_spend DESC, c.customer_id;
```

OUTPUT

customer_id	name	order_count	total_spend	review_count	last_order
5	Karthik Reddy	2	26296.00	2	2026-10-15 08:40:00
2	Priya Nair	2	22595.00	4	2026-09-20 13:15:00
3	Rohan Mehta	2	5993.00	1	2026-10-09 20:05:00
11	Imran Khan	1	4997.00	0	2026-10-22 17:55:00
1	Aarav Sharma	2	4696.00	3	2026-09-02 11:30:00
9	Arjun Das	1	4398.00	0	2026-09-27 10:50:00
10	Meera Kulkarni	1	4149.00	0	2026-10-03 15:25:00
8	Divya Menon	1	3797.00	2	2026-09-14 19:35:00
4	Ananya Iyer	1	3198.00	1	2026-08-19 18:22:00
6	Sneha Patil	1	2448.00	2	2026-09-06 16:45:00
7	Vikram Singh	1	0.00	0	2026-09-10 12:00:00
12	Lakshmi Venkatesh	0	0.00	0	NULL

(12 rows)

Explanation: Every column is a separate scalar subquery, so there is no join fan-out. Vikram Singh shows ₹0.00 spend because his only order was cancelled, and Lakshmi Venkatesh has no orders, so last_order is NULL.

06 SUMMARY

Choosing Between Joins and Subqueries

Need	Preferred approach	Reason
Show columns from several tables side by side	JOIN	Subqueries in WHERE cannot return extra columns
Keep rows that have no match	LEFT JOIN with IS NULL, or NOT EXISTS	INNER JOIN silently drops them
Compare with one overall value	Scalar subquery	Easy to read and avoids a self-join
Compare with a value for the same group	Correlated subquery	The inner query follows the outer row
Only check that related rows exist	EXISTS	Stops at the first match; no duplicate rows
Combine two separate aggregates	Derived tables joined together	Prevents multiplied rows and wrong totals

Note: Conclusion. The normalised design makes each fact live in exactly one place, and joins and subqueries rebuild any view of the data on demand. The same queries would still be correct if more customers, orders or products were added.